

University of Bahrain

College of Information Technology
Department of Computer Science
Course: Intelligent Systems (ITCS440)

Course Project Options – Semester 2nd 2025/2026

Submission date: (17/5) presentations date: (17/12 and 19/12)

Course project percentage representation (15%)

Submit 4 pages report describing the work you have done and the results obtained. Three students in one project.

Purpose of the Project

This project aims to let students apply Artificial Intelligence (AI) techniques — such as **rule-based reasoning, probabilistic inference, search, and constraint satisfaction** — using **Python programming**.

Students should demonstrate how AI techniques can be applied to real-world problems, using reasoning rather than advanced machine learning.

Each group must select **one project**, design and implement it using **Python**, test it on examples, and prepare a **written report**.

Project 1: Bayesian Network for Disease Prediction

Objective

Build a simple **Bayesian Network** that predicts the probability of a person having a specific disease (e.g., heart disease, flu, or diabetes) based on given symptoms and risk factors.

Background

Bayesian networks use **probabilistic reasoning** to model uncertain relationships between causes and effects (e.g., “smoking” increases the probability of “lung disease”).

Detailed Tasks

1. Choose a small set of variables (e.g., *age*, *smoking*, *exercise*, *blood pressure*, *disease*).
2. Design the **structure of the Bayesian network** — decide which variables are connected.
 - Example: Smoking → Disease
 - Age → Disease
 - Exercise → Disease
3. Define **conditional probability tables (CPTs)** manually or based on small datasets.
4. Write Python code using **pgmpy** or **pomegranate** to:
 - Define the network
 - Enter known evidence (e.g., "smoking=yes, exercise=no")
 - Compute the probability of having the disease
5. Test multiple cases and interpret the results.

Expected Output

- Console or graphical output showing disease probability for different inputs.
- Example:
- Given: Smoking=Yes, Exercise=No
- $P(\text{Disease=Yes}) = 0.78$

Tools: Python, pgmpy/pomegranate, pandas, numpy

Deliverables: Python code, screenshots, and explanation of reasoning.

Project 2: Rule-Based Chatbot for University FAQs

Objective

Develop a **rule-based chatbot** that answers common university-related questions (registration dates, course credit info, etc.) using AI reasoning rules.

Background

Rule-based chatbots rely on **pattern matching** and **IF-THEN logic** to decide how to respond, without using machine learning.

Detailed Tasks

1. Create a knowledge base of 20–30 rules like:
2. IF question contains "register" THEN reply "Registration opens on 5th January."
3. IF question contains "credit hours" THEN reply "Each course is 3 credit hours."
4. Write Python code that:
 - Takes user input as text
 - Matches it to rules
 - Displays the most appropriate response
5. Handle variations in questions (e.g., "How can I register?" or "When is registration?").
6. Optional: implement a simple **fallback rule** for unknown questions ("I'm not sure about that, please contact admin.").

Expected Output

An interactive terminal-based chatbot where the student can type and receive responses.
Example interaction:

User: When can I register for courses?

Bot: Registration opens on 5th January.

Tools: Python, regex, optional use of nltk for tokenization.

Deliverables: Python code, rule base file, example conversation transcript.

Project 3: Pathfinding Agent Using AI Search

Objective

Implement an AI agent that finds the **shortest path** between two points in a grid or maze using **BFS, DFS, and A*** algorithms.

Background

Search algorithms are core to AI problem-solving, used in robotics and navigation systems.

Detailed Tasks

1. Represent the grid as a **2D matrix** (0 = free cell, 1 = obstacle).
2. Implement the following algorithms:
 - **Breadth-First Search (BFS)**
 - **Depth-First Search (DFS)**
 - **A*** Search with a heuristic (e.g., Manhattan distance).
3. Compare:
 - Number of nodes explored
 - Execution time
 - Path length
4. Display the path found (text output or simple visualization).

Expected Output

Display the grid and highlight the found path.

Example:

S . . # .

. . . .

. # . . G

Path found using A*: [(0,0), (0,1), (1,1), (2,2), (2,4)]

Tools: Python, matplotlib or pygame for visualization.

Deliverables: Code, screenshots, performance comparison table.

Project 4: Expert System for Fault Diagnosis

Objective

Design an **expert system** to diagnose problems (e.g., car engine faults, computer issues, or mobile phone malfunctions) using rules.

Background

An expert system uses a **knowledge base** and an **inference engine** to make logical conclusions based on user inputs.

Detailed Tasks

1. Choose a small domain (e.g., car won't start).
2. Define 15–20 rules, e.g.:
3. IF engine_does_not_start AND battery_light_on THEN problem=battery_issue
4. IF engine_does_not_start AND fuel_gauge_empty THEN problem=fuel_issue
5. Implement **forward chaining**: start from known facts and apply rules to reach conclusions.
6. Ask the user questions interactively to gather facts.
7. Display the system's final conclusion or advice.

Expected Output

Example interaction:

System: Does the engine start? (yes/no)

User: no

System: Is the battery light on? (yes/no)

User: yes

System: Diagnosis: Battery issue.

Tools: Python only

Deliverables: Code, rule base, and example run.

Project 5: Game-Playing Agent (Tic-Tac-Toe)

Objective

Implement a **Tic-Tac-Toe game** with an AI opponent that uses **Minimax** (and optionally Alpha-Beta pruning) to make optimal moves.

Background

This project applies **adversarial search**, where the agent tries to maximize its own winning chances while minimizing the opponent's.

Detailed Tasks

1. Represent the 3x3 board using a list or matrix.
2. Implement functions:
 - Check win/draw
 - Generate possible moves
 - Evaluate board states
3. Implement the **Minimax algorithm** recursively.
4. Let the player play against the computer.
5. Optional: add random or easy mode to compare with the smart AI.

Expected Output

A playable console game:

User: X | AI: O

Final Board:

X | O | X

O | X | O

Winner: Player

Tools: Python, optional pygame for GUI.

Deliverables: Source code, game screenshots, short evaluation.

Project 6: Intelligent Scheduling Assistant

Objective

Develop a scheduling system that automatically assigns tasks or classes to available time slots based on **constraints**.

Background

Constraint Satisfaction Problems (CSPs) involve assigning values to variables under given restrictions — common in timetabling and planning.

Detailed Tasks

1. Define variables (e.g., courses or tasks) and domains (available times).
2. Define constraints, e.g.:
 - "No two courses at the same time"
 - "Instructor A not available on Monday"
3. Implement a **backtracking search algorithm** to find a valid assignment.
4. Display the generated schedule neatly.

Expected Output

Example:

Course A → Monday 10:00

Course B → Tuesday 12:00

No conflicts detected.

Tools: Python

Deliverables: Code, example run, explanation of constraints.

Project 7: Knowledge-Based Recommendation System

Objective

Develop a simple rule-based system that recommends items (e.g., movies, products, or courses) based on user preferences.

Background

Unlike machine learning recommenders, knowledge-based systems use explicit logic like: "If the user likes comedies and short movies → recommend X."

Detailed Tasks

1. Define attributes (genre, duration, level, etc.).
2. Create 15–20 rules for recommendations.

3. Ask the user for preferences interactively.
4. Use logical reasoning to select the best recommendation.

Expected Output

User: I like action and short movies.

System: I recommend “John Wick (2014)”.

Tools: Python

Deliverables: Code, rule set, example interactions.

Project 8: ML Applications

Objective

Apply various ML on a certain dataset

Background

Various ML techniques and how to apply them

Detailed Tasks

Select any suitable dataset from UCI, GitHub or Kaggle for a specific purpose (for example, for medical diseases classifications, Emotion classification through text, SPAM article classifications, various types of plants classification such as Mushroom, sound classification, etc.). Try various ML models such as regression, Neural networks, SVM, Decision Trees and Random Forest and compare your obtained results with very good analysis. Apply data preprocessing before applying the ML models.

Expected Output

A comparative results between the used ML models showing the results of Accuracy, MSE, F1 Score.

Tools: Python

Evaluation Rubric

Component	Description	Weight
Understanding & Design	Clear problem definition and design	20%

Implementation	Correct, efficient, and modular code	35%
Application of AI Concepts	Demonstrated understanding of reasoning/inference	25%
Report & Presentation	Clarity, format, and analysis	20%